

Métodos Numéricos

con Python

Manual Tecnico

Universidad Mariano Galvez de Guatemala

Facultad de Ingenieria en Sistemas

Curso: Métodos Numéricos

Desarrollado por: Luis Pedro Ramirez Segura y Gerson Samuel Guzman Franco

Tabla de Contenidos

1. Arquitectura del Sistema
 - 1.1 Estructura de Directorios
 - 1.2 Patrón de Diseño
 - 1.3 Diagrama de Módulos
2. Flujo de la Aplicación
 - 2.1 Ciclo de Vida de una Vista
 - 2.2 Flujo de Cálculo
3. Algoritmos Implementados
 - 3.1 Cálculo de Raíces
 - 3.2 Interpolación
 - 3.3 Sistemas de Ecuaciones
 - 3.4 Integración Numérica
 - 3.5 Ecuaciones Diferenciales
4. Decisiones de Diseño
5. Sistema de Temas Visuales
6. Módulo de Exportación PDF
7. Problemas y Soluciones
8. Referencias Bibliográficas

1. Arquitectura del Sistema

La aplicación sigue una arquitectura en capas con separación clara entre lógica de negocio (algoritmos), presentación (vistas) y utilidades compartidas. La interfaz gráfica se construye sobre CustomTkinter, una extensión moderna de Tkinter.

1.1 Estructura de Directorios

```
proyecto_metodos_fixed/
■■■ main.py                Punto de entrada, App principal, navegación
■■■ requirements.txt       Dependencias del proyecto
■■■ instalar_y_ejecutar.py Script de instalación automática
■■■ ui/
■   ■■■ themes.py         Definición de paletas de color (3 temas)
■   ■■■ components.py     Componentes reutilizables (botones, banners)
■   ■■■ ayuda_formulas.py Ventana de ayuda con fórmulas matemáticas ■■■
raices/
■   ■■■ biseccion.py       Algoritmo de Bisección
■   ■■■ newton_raphson.py  Algoritmo de Newton-Raphson + cálculo simbólico
■   ■■■ secante.py         Algoritmo de la Secante
■   ■■■ biseccion_view.py  Vista de Bisección
■   ■■■ newton_raphson_view.py Vista de Newton-Raphson
```

Capa	Archivos	Responsabilidad
Model (Algoritmos)	raices/*.py, interpolacion/*.py, etc.	Lógica matemática pura, sin UI

View (Vistas)	*_view.py	Interfaz gráfica, entrada/salida
■ ■■■■ secante_view.py	Vista de la Secante	
■ ■■■■ raices_view.py	Vista contenedora (selector de método) ■■■■	
interpolacion/		
■ ■■■■ lagrange.py	Algoritmo de Lagrange	
■ ■■■■ newton_interp.py	Diferencias divididas de Newton	
■ ■■■■ interp_view.py	Vista de Interpolación ■■■■	
sistemas/		
■ ■■■■ gauss_seidel.py	Método de Gauss-Seidel	
■ ■■■■ factorizacion_lu.py	Factorización LU (Doolittle)	
■ ■■■■ sistemas_view.py	Vista de Sistemas	
■■■■ integracion/		
■ ■■■■ metodos.py	Trapezio, Simpson 1/3, Diferencias Finitas	
■ ■■■■ integracion_view.py	Vista de Integración ■■■■	
diferenciales/		
■ ■■■■ metodos_edo.py	Euler, Runge-Kutta 4	
■ ■■■■ diferenciales_view.py	Vista de EDOs	
■■■■ export_pdf/		
■ ■■■■ exporter.py	Generador de PDF (ReportLab) ■■■■	
utils/		
■■■■ validators.py	Validación de funciones, errores numéricos	

1.2 Patrón de Diseño

Se aplica el patrón **MVC simplificado** (Model-View) con las siguientes capas:

UI Shared	ui/themes.py, ui/components.py	Estilos y componentes reutilizables
Utils	utils/validators.py	Parseo de funciones, cálculo de errores

1.3 Diagrama de Módulos (Dependencias)

El siguiente diagrama muestra las dependencias entre módulos principales: `main.py`

```

■■■■ ui/themes.py           (importado por todas las vistas)
■■■■ ui/components.py       (importado por todas las vistas)
■■■■ raices/raices_view.py
■   ■■■■ raices/biseccion_view.py → raices/biseccion.py
■   ■■■■ raices/newton_raphson_view.py → raices/newton_raphson.py
■   ■■■■ raices/secante_view.py → raices/secante.py
■■■■ interpolacion/interp_view.py
■   ■■■■ interpolacion/lagrange.py
■   ■■■■ interpolacion/newton_interp.py
■■■■ sistemas/sistemas_view.py
■   ■■■■ sistemas/gauss_seidel.py
■   ■■■■ sistemas/factorizacion_lu.py
■■■■ integracion/integracion_view.py → integracion/metodos.py
■■■■ diferenciales/diferenciales_view.py → diferenciales/metodos_edo.py
■■■■ export_pdf/exporter.py      (invocado desde todas las vistas)

```

2. Flujo de la Aplicación

2.1 Ciclo de Vida de una Vista

Cada módulo sigue el mismo ciclo de vida gestionado por la clase App en main.py:

Fase	Descripción
1. Navegación	El usuario hace clic en un ítem del sidebar → <code>App._navigate(key)</code>
2. Instanciación	Si la vista no existe, se crea: <code>cls(self.content, export_callback=...)</code>
3. Presentación	Se llama <code>view.pack(fill='both', expand=True)</code>
4. Cálculo	El usuario ingresa datos → presiona Calcular → <code>view._calcular()</code>
5. Resultado	Se actualiza <code>ResultadoBox</code> , tabla y gráfica <code>matplotlib</code> embebida
6. Exportación	Botón Exportar → <code>export_callback(result, fig, modulo)</code>
7. Cambio tema	<code>App._change_theme()</code> destruye todas las vistas y las recrea

2.2 Flujo de Cálculo (Diagrama de Flujo Textual)

```
[Usuario ingresa f(x) y parámetros]
|
v
[validators.parse_function(fx_str)]
|           |
ERROR       función compilada
|           |
[mostrar err] v
              [llamada al algoritmo]
              |
          ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
          |                                 |
[ÉXITO]                               [EXCEPCIÓN]
          |                   | [result dict]:
[mostrar error]
- raiz / resultado
- tabla (iteraciones)
- procedimiento (texto) - figura matplotlib      |           v [ResultadoBox.set_text(procedimiento)]
[actualizar Treeview con tabla]
[canvas.draw() con figura]
|
v
[Botón PDF disponible]
```

3. Algoritmos Implementados

3.1 Cálculo de Raíces

Bisección — biseccion.py

Divide iterativamente el intervalo $[a, b]$ por la mitad. Requiere $f(a) \cdot f(b) < 0$ (Teorema de Bolzano). Converge garantizadamente a una tasa lineal.

Fórmula: $c = (a + b) / 2$

Si $f(c) = 0 \rightarrow$ raíz exacta. Si $f(a) \cdot f(c) < 0 \rightarrow b = c$. Si no $\rightarrow a = c$. Criterio de parada: $|b - a| < \text{tolerancia}$ o $|f(c)| < \text{tolerancia}$.

Newton-Raphson — newton_raphson.py

Método de convergencia cuadrática que usa la tangente a la curva. Requiere que $f'(x) \neq 0$ en el punto. Utiliza SymPy para calcular la derivada simbólicamente a partir del string de $f(x)$.

Fórmula: $x(n+1) = x(n) - f(x(n)) / f'(x(n))$

Secante — secante.py

Similar a Newton-Raphson pero aproxima la derivada usando dos puntos anteriores. No requiere calcular $f'(x)$.

Convergencia superlineal (orden ≈ 1.618 , número áureo).

Fórmula: $x(n+1) = x(n) - f(x(n)) * (x(n) - x(n-1)) / (f(x(n)) - f(x(n-1)))$

3.2 Interpolación

Lagrange — lagrange.py

Construye el polinomio interpolante usando los polinomios base de Lagrange $L_i(x)$. Para $n+1$ puntos, el polinomio resultante tiene grado n . No requiere resolver un sistema.

$L_i(x) = \text{PRODUCTO}((x - x_j) / (x_i - x_j))$ para $j \neq i$

Newton (Diferencias Divididas) — newton_interp.py

Construye el polinomio usando diferencias divididas. Permite agregar puntos incrementalmente sin recalculr todo el polinomio. La tabla de diferencias divididas se muestra en el procedimiento.

$f[x_0, x_1, \dots, x_k] = (f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]) / (x_k - x_0)$

3.3 Sistemas de Ecuaciones

Gauss-Seidel — gauss_seidel.py

Método iterativo para $Ax = b$. En cada iteración actualiza cada componente x_i usando los valores más recientes. Requiere matriz diagonalmente dominante para garantizar convergencia. Verifica esta condición e informa al usuario.

Factorización LU (Doolittle) — factorizacion_lu.py

Método directo que factoriza $A = L \cdot U$ donde L es triangular inferior (diagonal=1) y U es triangular superior.

Resuelve $Ly = b$ (sustitución progresiva) y luego $Ux = y$ (sustitución regresiva).

3.4 Integración Numérica

Método	Fórmula base	Error $O(h)$
Punto Medio	$h * \sum f((x_i + x_{i+1})/2)$	$O(h^2)$
Trapezio	$h/2 * (f(a) + 2 * \sum f(x_i) + f(b))$	$O(h^2)$
Simpson 1/3	$h/3 * (f(a) + 4f(x_1) + 2f(x_2) + \dots + f(b))$	$O(h^4)$
Dif. Finitas	$f'(x) \approx (f(x+h) - f(x-h)) / (2h)$	$O(h^2)$

El módulo calcula además el valor real usando `scipy.integrate.quad` para comparar el resultado numérico y calcular errores absoluto, relativo y porcentual.

3.5 Ecuaciones Diferenciales

Método de Euler — `metodos_edo.py`

Método explícito de primer orden. Simple pero con error acumulado $O(h)$. Apropiado para pasos pequeños h .

```
y(n+1) = y(n) + h * f(x(n), y(n))
```

Runge-Kutta 4 (RK4) — `metodos_edo.py`

Método de cuarto orden con error $O(h^4)$ por paso. Calcula cuatro pendientes k_1, k_2, k_3, k_4 y las promedia con pesos. Considerablemente más preciso que Euler para el mismo tamaño de paso.

```
k1=h*f(x,y) k2=h*f(x+h/2, y+k1/2) k3=h*f(x+h/2, y+k2/2) k4=h*f(x+h, y+k3)
```

```
y(n+1) = y(n) + (k1 + 2*k2 + 2*k3 + k4) / 6
```

4. Decisiones de Diseño

Uso de CustomTkinter

Se eligió CustomTkinter por su soporte de temas oscuro/claro nativos, widgets modernos y compatibilidad crossplatform sin dependencias adicionales pesadas.

Separación algoritmo-vista

Los algoritmos (`biseccion.py`, `gauss_seidel.py`, etc.) son funciones puras sin referencias a la UI. Esto facilita el testing unitario y la reutilización del código.

Evaluación dinámica de funciones

La función `parse_function()` en `validators.py` compila el string $f(x)$ del usuario con `compile()` + `eval()` en un namespace seguro que incluye `numpy` y `math`. Se capturan excepciones de compilación y evaluación.

Derivación simbólica con SymPy

Newton-Raphson requiere $f'(x)$. En lugar de pedir al usuario que la ingrese manualmente, se usa SymPy para derivar simbólicamente el string $f(x)$, mejorando la usabilidad.

Lazy loading de vistas

Las vistas se instancian solo cuando el usuario navega a ese módulo por primera vez (lazy loading). Esto reduce el tiempo de arranque y el uso de memoria.

Sistema de callbacks para exportación

La función `export_callback` se pasa desde App a cada vista al crearla. Esto desacopla la lógica de guardado de archivos (cuadros de diálogo del OS) de la lógica de generación del PDF.

5. Sistema de Temas Visuales

El sistema de temas se gestiona en `ui/themes.py`. Cada tema es un diccionario de claves semánticas mapeadas a colores hexadecimales. Las vistas acceden al tema mediante `get_theme()` que retorna el diccionario del tema activo.

Clave	Uso
<code>bg_main</code>	Color de fondo principal del área de contenido
<code>bg_sidebar</code>	Color del panel lateral y header/footer
<code>bg_input</code>	Fondo de campos de entrada
<code>accent</code>	Color de acento (botón primario, radio activo)
<code>accent_hover</code>	Acento en hover
<code>text_primary</code>	Texto principal
<code>text_secondary</code>	Texto secundario / etiquetas
<code>text_accent</code>	Texto del ítem activo en sidebar
<code>sidebar_active</code>	Fondo del ítem activo en sidebar
<code>sidebar_active_bar</code>	Barra lateral del ítem activo
<code>btn_warning</code>	Color del botón de advertencia (amarillo)
<code>border</code>	Color de bordes y separadores

Al cambiar de tema, `App._change_theme()` llama `set_theme(name)`, destruye todas las vistas del caché y las recrea. Los frames estructurales (`header`, `sidebar`, `footer`) se reconfiguran directamente con `configure(fg_color=...)`.

6. Módulo de Exportación PDF

El módulo `export_pdf/exporter.py` utiliza la librería `ReportLab` para generar documentos PDF profesionales. La función principal `exportar_pdf()` recibe:

Parámetro	Tipo	Descripción
<code>metodo</code>	<code>str</code>	Nombre del método numérico
<code>datos_entrada</code>	<code>dict</code>	Pares clave-valor de entradas del usuario
<code>resultado_str</code>	<code>str</code>	Resumen del resultado principal
<code>tabla_headers</code>	<code>list[str]</code>	Encabezados de la tabla de iteraciones
<code>tabla_rows</code>	<code>list[list]</code>	Filas de la tabla (máx 100)
<code>procedimiento</code>	<code>str</code>	Texto completo del procedimiento paso a paso
<code>fig</code>	<code>Figure None</code>	Figura matplotlib (se convierte a imagen PNG)
<code>nombre_archivo</code>	<code>str</code>	Ruta completa del archivo de salida .pdf
<code>desarrolladores</code>	<code>tuple</code>	Nombres para el pie de página

La figura matplotlib se serializa a bytes PNG en memoria usando `io.BytesIO + fig.savefig(buf, format='png')`, luego se embebe en el PDF usando `reportlab.platypus.Image`. Esto evita escribir archivos temporales en disco.

7. Problemas Encontrados y Soluciones Aplicadas

Problema: Cambio de tema no se aplicaba completamente

Solución: Al cambiar de tema, los widgets hijos mantenían los colores anteriores. Solución: destruir todas las vistas del caché y recrearlas desde cero. Los frames estructurales se reconfiguran directamente.

Problema: División por cero en Secante y Newton-Raphson

Solución: Si $f'(x) = 0$ o $f(x_1) - f(x_0) \approx 0$, ocurría `ZeroDivisionError` no controlado. Solución: verificar el denominador antes de dividir y lanzar `ZeroDivisionError` con mensaje descriptivo que la vista captura y muestra al usuario.

Problema: Módulos no cargaban después de cambio de tema

Solución: `winfo_exists()` retornaba `True` para widgets destruidos en algunos casos. Solución: envolver la verificación en `try/except` y forzar recreación si falla.

Problema: SymPy lento en derivación de funciones complejas

Solución: Para funciones largas, `sympy.diff()` podía tardar varios segundos. Solución: la derivación ocurre solo al presionar Calcular, con manejo de excepción que sugiere ingresar la derivada manualmente si falla.

Problema: Tabla de iteraciones lenta con muchas filas

Solución: Tkinter Treeview se vuelve lento con más de 500 filas. Solución: limitar la tabla a las primeras 100 iteraciones en la vista y exportar todas las iteraciones en el PDF (hasta 100).

8. Referencias Bibliográficas

[1] Chapra, S. C. & Canale, R. P. (2015). *Métodos Numéricos para Ingenieros* (7a ed.). McGraw-Hill.

[2] Burden, R. L. & Faires, J. D. (2011). *Numerical Analysis* (9th ed.). Brooks/Cole, Cengage Learning.

[3] Meurer, A. et al. (2017). SymPy: symbolic computing in Python. *PeerJ Computer Science*, 3, e103.

<https://doi.org/10.7717/peerj-cs.103>

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.

[5] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362.

<https://doi.org/10.1038/s41586-020-2649-2>

[6] Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261-272.

[7] CustomTkinter Documentation. (2023). <https://customtkinter.tomschimansky.com/>

[8] ReportLab PDF Library User Guide. (2023). <https://docs.reportlab.com/>

[9] Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>